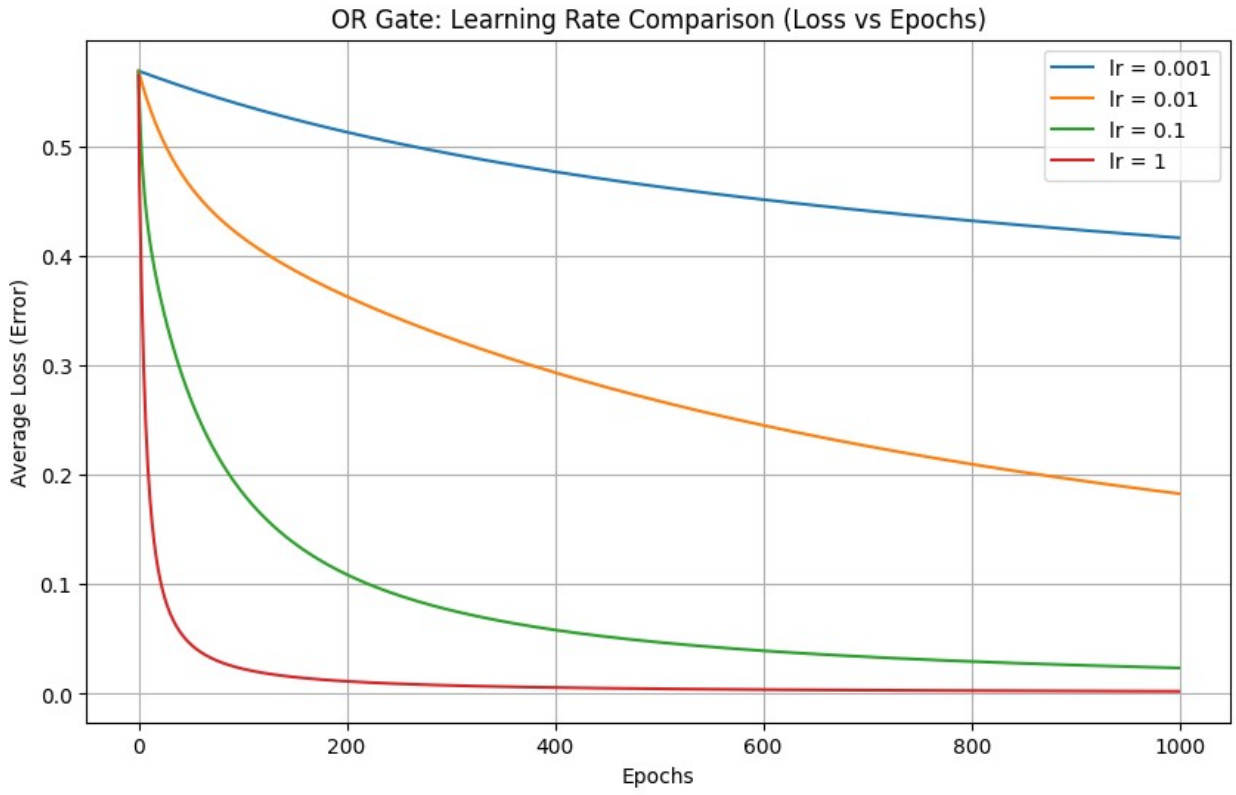



```
Weights: [6.7832796  6.78742098]
Bias: -2.9139465005185365
Prem Massand,431
Date:05/02/2026
```

LAB 4| 12/02/2026

```
import pandas as pd

df = pd.read_csv("spam_or_not_spam.csv")

print("Shape:", df.shape)
print(df.head())
print(df["label"].value_counts())

Shape: (3000, 2)
                                               email  label
0   date wed NUMBER aug NUMBER NUMBER NUMBER NUMB...      0
1  martin a posted tassos papadopoulos the greek ...      0
2  man threatens explosion in moscow thursday aug...      0
3  klez the virus that won t die already the most...      0
4   in adding cream to spaghetti carbonara which ...      0
label
0    2500
```

```
1     500
Name: count, dtype: int64
```

```
from sklearn.model_selection import train_test_split

df = df.dropna(subset=["email"])

X = df["email"]
y = df["label"].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Train size:", len(X_train))
print("Test size:", len(X_test))
```

```
Train size: 2399
Test size: 600
```

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(stop_words="english", max_features=5000)

X_train_vec = vectorizer.fit_transform(X_train).toarray()
X_test_vec = vectorizer.transform(X_test).toarray()

print("Train vector shape:", X_train_vec.shape)
print("Test vector shape:", X_test_vec.shape)
```

```
Train vector shape: (2399, 5000)
Test vector shape: (600, 5000)
```

```
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def binary_cross_entropy(y_true, y_pred):
    eps = 1e-9
    y_pred = np.clip(y_pred, eps, 1 - eps)
    return -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1
- y_pred))

def train_single_neuron(X_train, y_train, X_test, y_test, lr=0.1,
epochs=50):
```

```
    n_features = X_train.shape[1]
    w = np.zeros(n_features)
    b = 0

    losses = []
    train_accs = []

    for epoch in range(epochs):


        z = np.dot(X_train, w) + b
        y_hat = sigmoid(z)


        loss = binary_cross_entropy(y_train, y_hat)


        y_pred_train = (y_hat >= 0.5).astype(int)
        train_acc = np.mean(y_pred_train == y_train)


        dw = np.dot(X_train.T, (y_hat - y_train)) / len(y_train)
        db = np.mean(y_hat - y_train)


        w = w - lr * dw
        b = b - lr * db

        losses.append(loss)
        train_accs.append(train_acc)


    z_test = np.dot(X_test, w) + b
    y_hat_test = sigmoid(z_test)
    y_pred_test = (y_hat_test >= 0.5).astype(int)
    test_acc = np.mean(y_pred_test == y_test)

    return losses, train_accs, test_acc

z_test = np.dot(X_test_vec, w) + b
y_hat_test = sigmoid(z_test)

y_pred_test = (y_hat_test >= 0.5).astype(int)

test_acc = np.mean(y_pred_test == y_test)
print("Test Accuracy:", test_acc)
```

```
Test Accuracy: 0.8333333333333334
```

```
from sklearn.metrics import precision_score, recall_score, f1_score

precision = precision_score(y_test, y_pred_test)
```

```
recall = recall_score(y_test, y_pred_test)
f1 = f1_score(y_test, y_pred_test)

print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
```

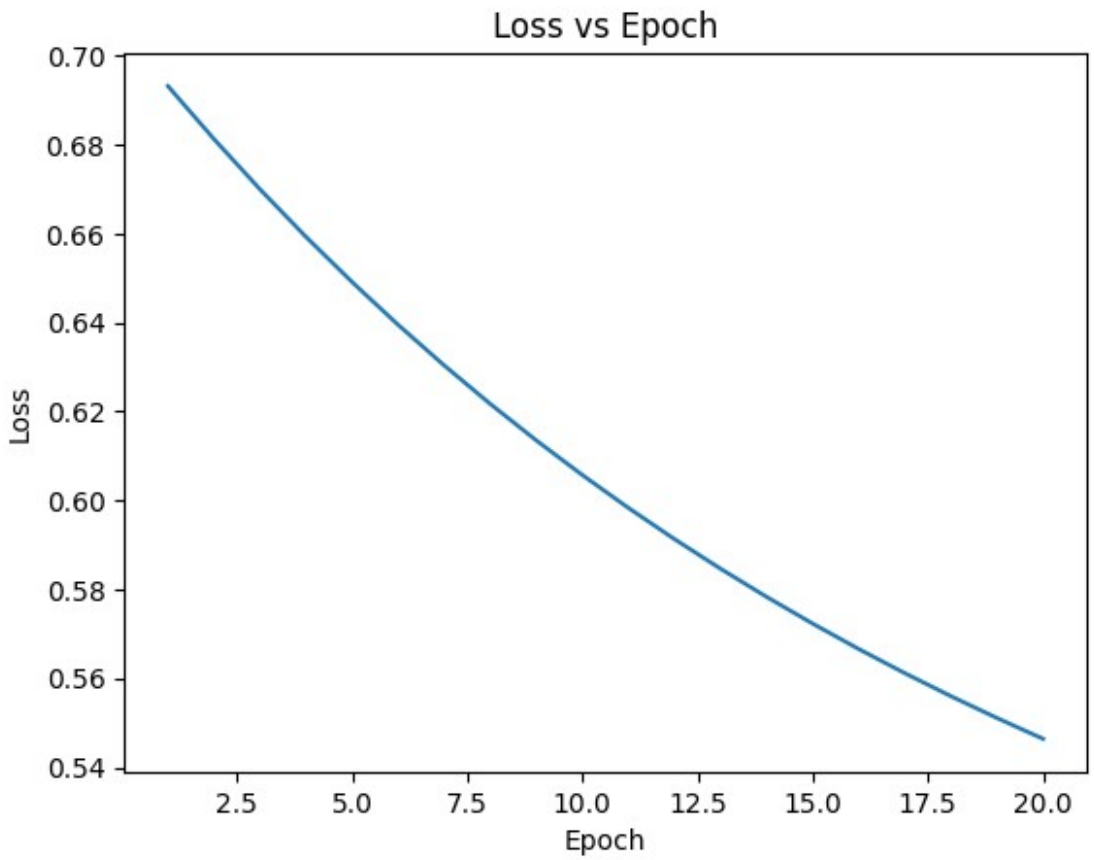
```
Precision: 0.0
Recall: 0.0
F1 Score: 0.0
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/
_classification.py:1565: UndefinedMetricWarning: Precision is ill-
defined and being set to 0.0 due to no predicted samples. Use
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))
```

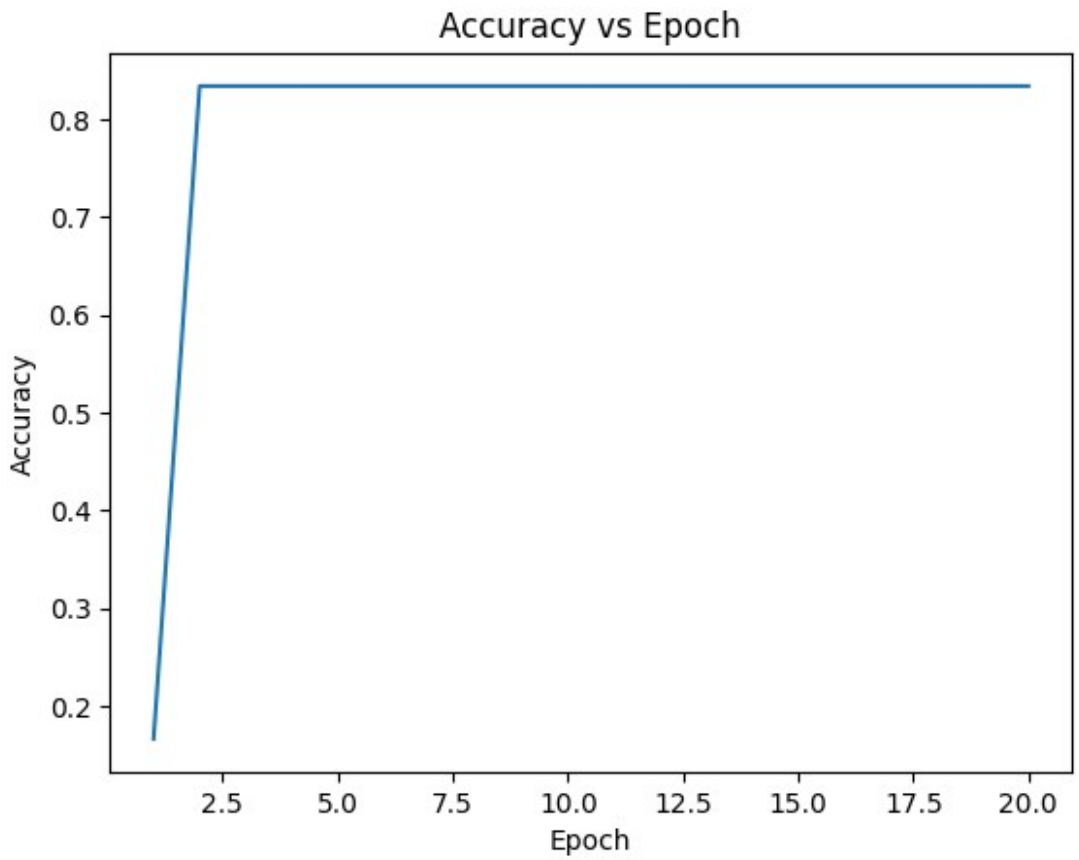
```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred_test)
print("Confusion Matrix:\n", cm)
```

```
Confusion Matrix:
 [[500   0]
 [100   0]]
```

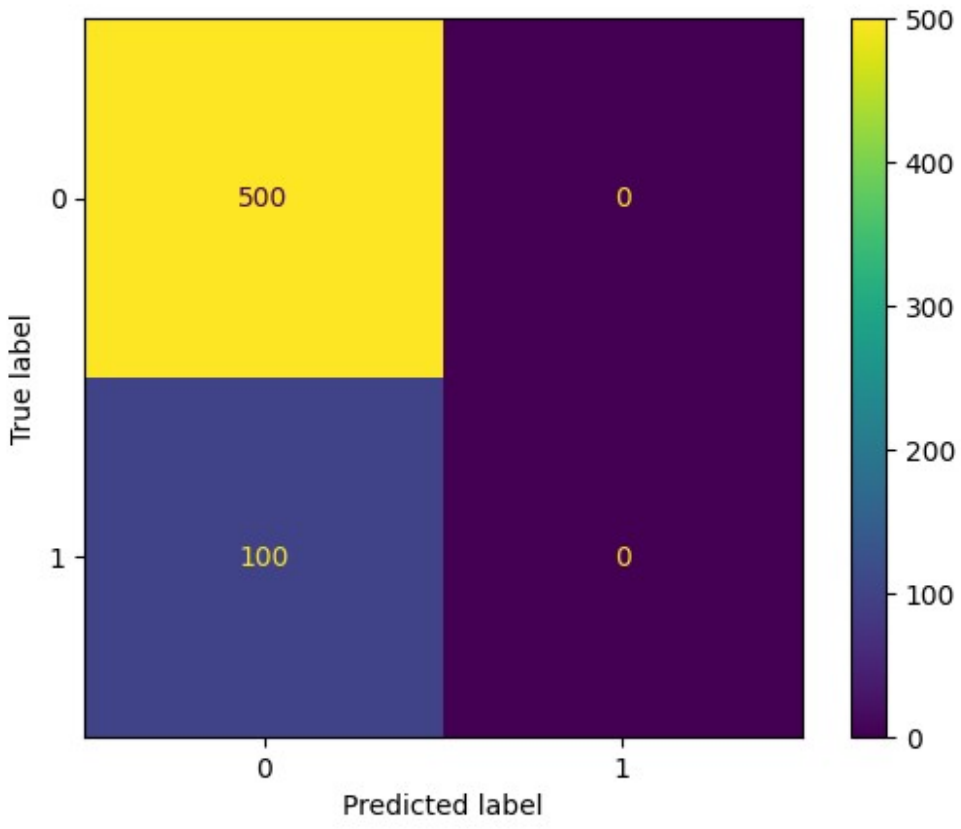
```
import matplotlib.pyplot as plt

plt.plot(range(1, epochs+1), losses)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Loss vs Epoch")
plt.show()
```



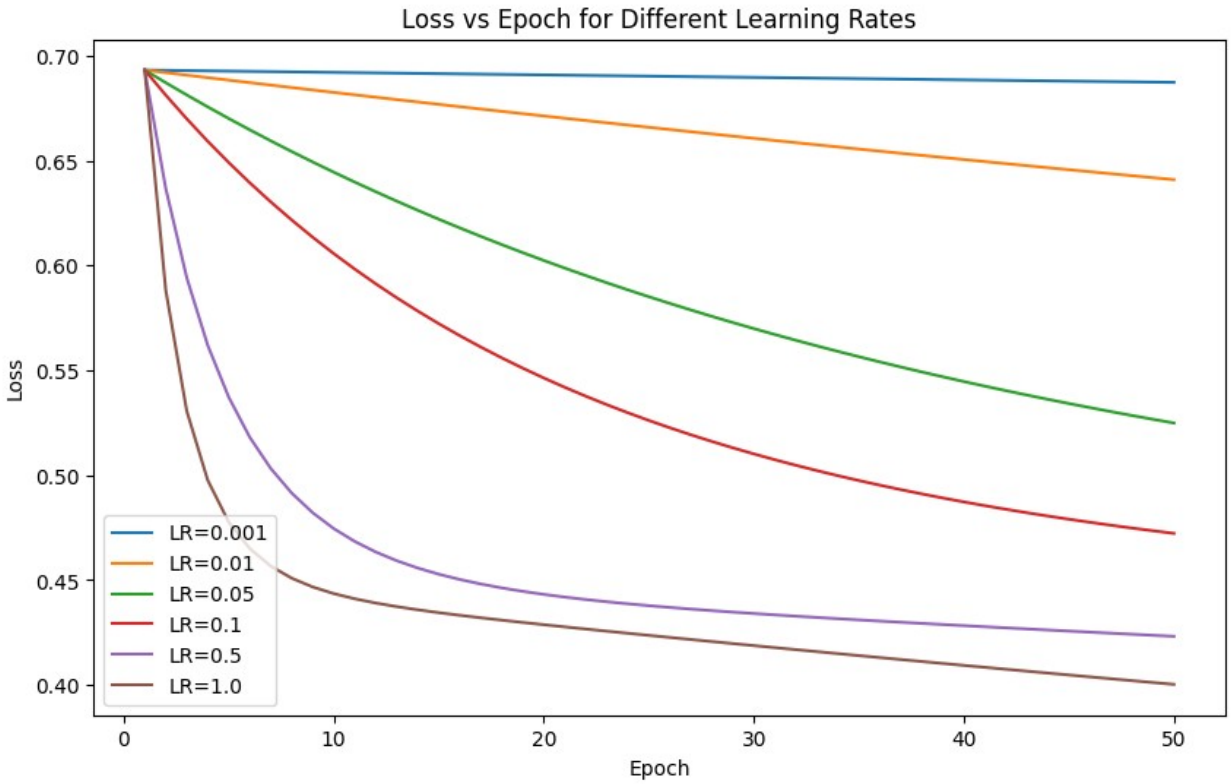

```
plt.plot(range(1, epochs+1), accs)
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Accuracy vs Epoch")
plt.show()
```



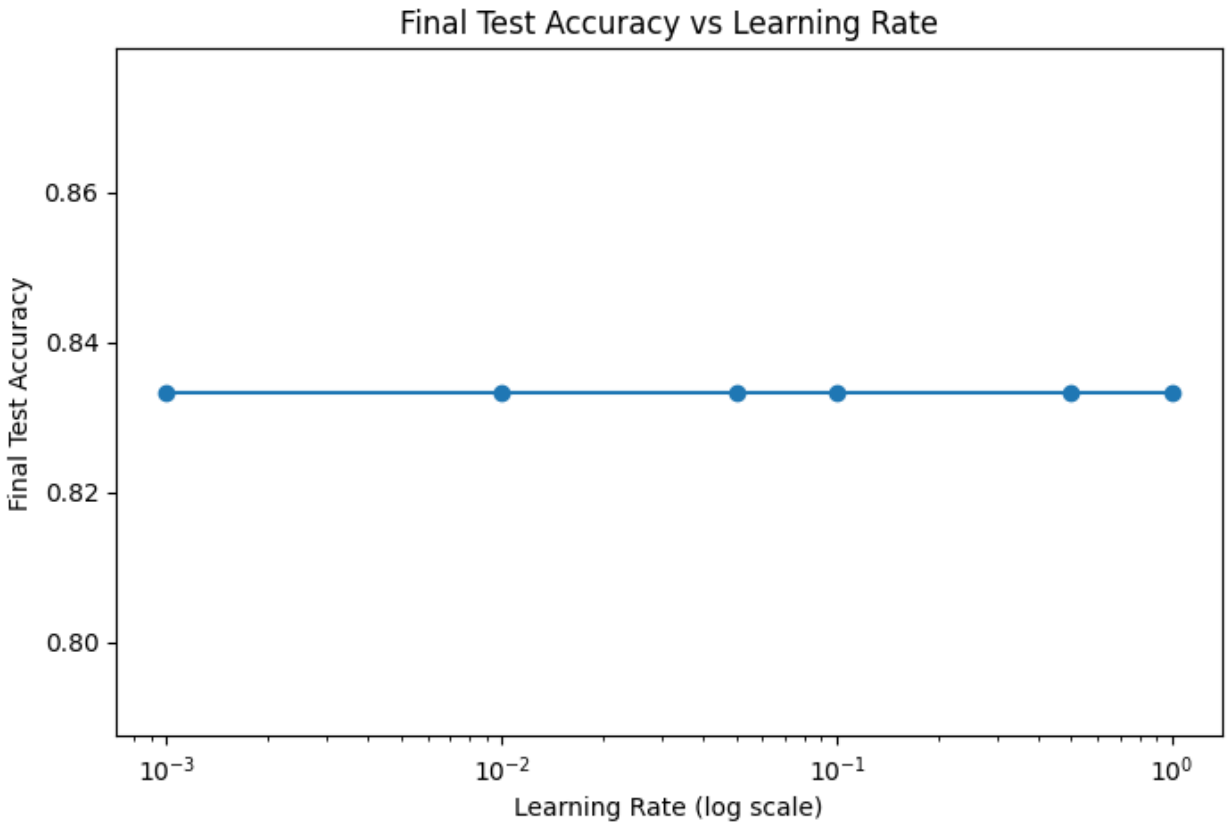

```
from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt

disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
plt.show()
```




```
learning_rates = [0.001, 0.01, 0.05, 0.1, 0.5, 1.0]
epochs = 50

final_test_acc = {}

plt.figure(figsize=(10,6))

for lr in learning_rates:
    losses, train_accs, test_acc = train_single_neuron(
        X_train_vec, y_train, X_test_vec, y_test,
        lr=lr, epochs=epochs
    )

    final_test_acc[lr] = test_acc
    plt.plot(range(1, epochs+1), losses, label=f"LR={lr}")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Loss vs Epoch for Different Learning Rates")
plt.legend()
plt.show()
```




```
lrs = list(final_test_acc.keys())
accs = list(final_test_acc.values())

plt.figure(figsize=(8,5))
plt.plot(lrs, accs, marker="o")

plt.xscale("log")  # IMPORTANT (because LR values are exponential)
plt.xlabel("Learning Rate (log scale)")
plt.ylabel("Final Test Accuracy")
plt.title("Final Test Accuracy vs Learning Rate")
plt.show()

print("Final Test Accuracy for each LR:")
for lr, acc in final_test_acc.items():
    print(f"LR={lr}  ->  Test Accuracy={acc:.4f}")
```




```
Final Test Accuracy for each LR:
LR=0.001  ->  Test Accuracy=0.8333
LR=0.01  ->  Test Accuracy=0.8333
LR=0.05  ->  Test Accuracy=0.8333
LR=0.1  ->  Test Accuracy=0.8333
LR=0.5  ->  Test Accuracy=0.8333
LR=1.0  ->  Test Accuracy=0.8333
```

LAB 5

```python
import numpy as np

# XOR dataset (4 input combinations)
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

# XOR output
y = np.array([
```